

PR Pre-Flight Guardian

An autonomous Node.js CLI that chains static security auditing, automated unit test verification, and PR documentation generation into a single unified pre-flight command before code merges.

DEVELOPER / TEAM

Qaasim-Falade Abdullah

AI PLATFORM

IBM Bob IDE (Agent Mode 2.0)

GITHUB REPOSITORY

Alqaasimee-jr2/ibm-coding-challenge

BOBCOIN EFFICIENCY

40 / 40 Bobcoins Allocation

The 45–Minute Pull Request Bottleneck

DEVELOPER FRICTION

QUALITY GAPS

⚠ Where Developer Hours Disappear

The pull request boundary is the most critical quality gate in software delivery, yet it remains manual, error-prone, and heavily fragmented:

- **Manual Code Reviews:** Developers spend 15 minutes eyeballing files for leaked keys, dynamic eval(), or SQL strings.
- **Rushed Boilerplate Tests:** Under sprint deadlines, developers spend 20 minutes authoring tests, skipping boundary cases.
- **Manual PR Documentation:** Copy-pasting terminal logs and guessing at security risk profiles takes 10+ minutes.
- **Costly Context Switching:** Developers juggle linters, test runners, git diffs, and web portals simultaneously.

🚨 The High Cost of Unchecked PRs

When manual auditing fails under pressure, critical vulnerabilities escape directly into production branches:

- **Leaked High-Entropy Secrets:** Hardcoded Stripe live keys and JWT secrets committed into Git history.
- **Dangerous Dynamic Code:** Unescaped eval() calls and SQL string concatenations open remote execution vectors.
- **Untested Edge Cases:** Untested business logic (e.g. stacked volume discounts) leads to revenue loss.
- **Reviewer Fatigue:** Sparse PR summaries force reviewers to guess intent, delaying merges for days.

Before vs. After Guardian

90X FASTER

ZERO-FRICTION

BEFORE GUARDIAN

45 MINUTES

- **15 Min Security Check:** Eyeballing code diffs, searching for secrets.
- **20 Min Unit Testing:** Manually writing Jest tests for new methods.
- **10 Min PR Summary:** Manually typing pull request notes and checklist.
- **High Cognitive Load:** Juggling multiple tools and terminals.
- **Human Error:** Silent leaks, missed boundary tests, broken builds.

AFTER GUARDIAN

30 SECONDS

- **Autonomous Static Scan:** AST pattern matching with CVSS 3.1 scores.
- **Automated Test Execution:** Regression suites assert 100% green pass rate.
- **Instant PR Markdown:** Complete risk matrix & reviewer checklist generated.
- **Interactive HTML Report:** Standalone dashboard with readiness scorecard.
- **CI/CD Gate Ready:** Exit code 1 blocker verdict stops unsafe merges.

45m → 30s
TIME SAVED PER PR

100%
AUTOMATED COVERAGE

0
RUNTIME DEPENDENCIES

1 Command
NODE GUARDIAN.JS

Chained 4-Phase Pre-Flight Architecture

DETERMINISTIC

CHAINED TASKS

Phase 1: Static Security Scanner (AST & Regex)

Scans target files for hardcoded API keys, JWT secrets, SQL injection concatenations, and unsafe `dynamic eval()`. Generates `findings.json` with CVSS 3.1 base scores & suggested patches.

Phase 2: Automated Test Execution

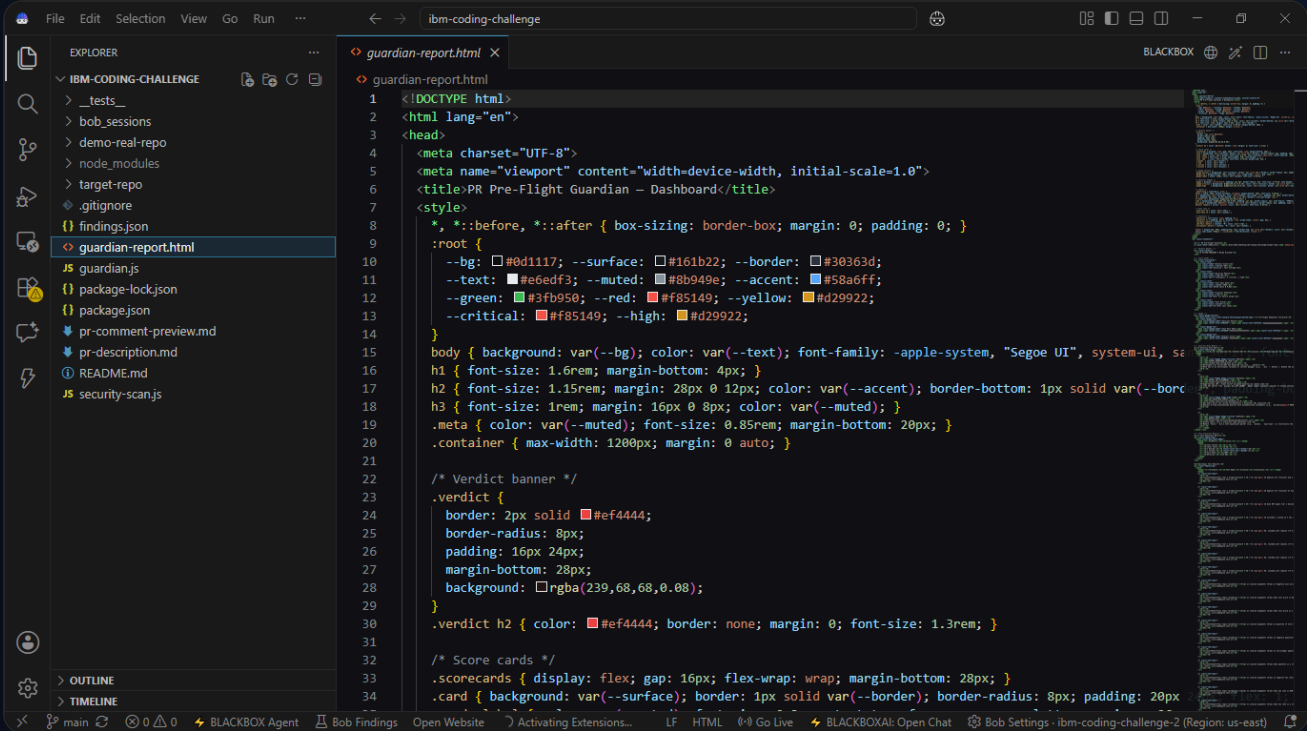
Executes Jest regression suites across complex pricing logic (tiered volume discounts, loyalty bonus stacking, boundary validation), verifying 100% green pass rates.

Phase 3: PR Markdown Documentation Synthesis

Merges security findings and test output into GitHub/GitLab-ready pull request summaries (`pr-description.md` & `pr-comment-preview.md`) with a risk matrix & sign-off checklist.

Phase 4: Interactive HTML Dashboard

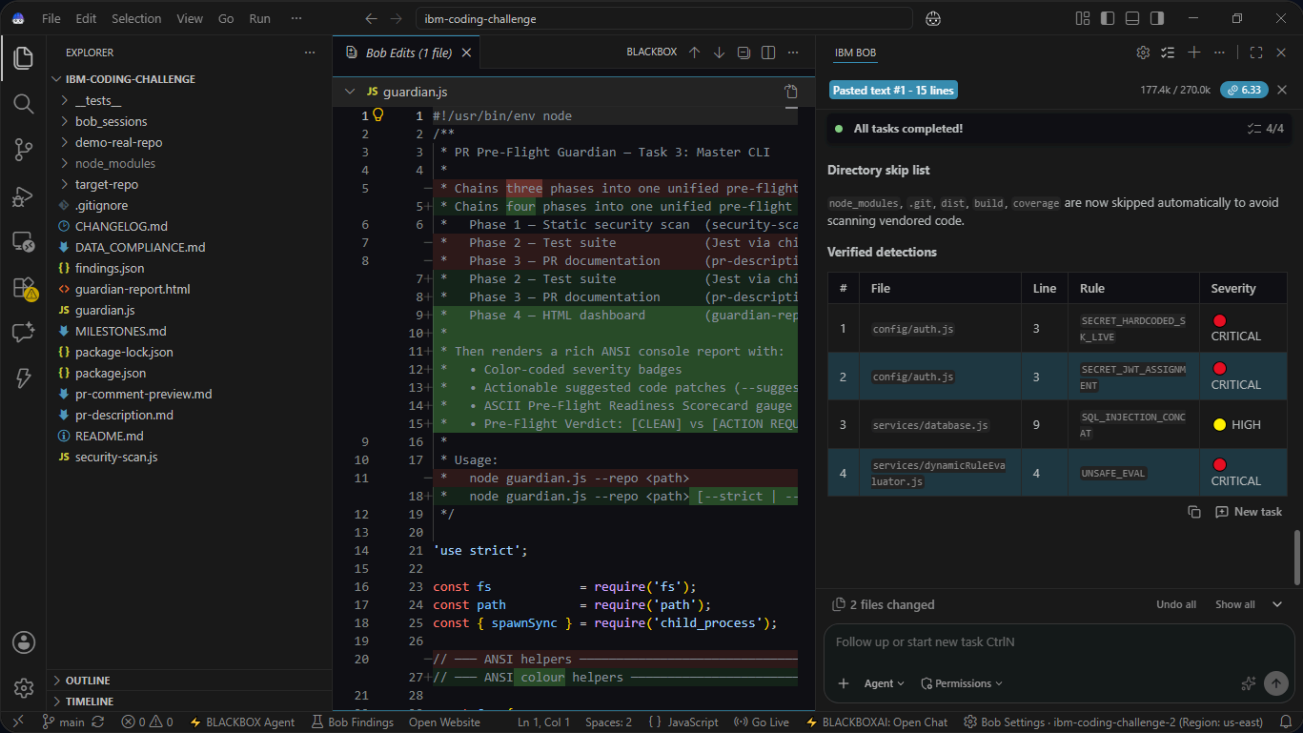
Renders self-contained dark-mode report (`guardian-report.html`) with filterable findings, interactive tabs, and visual pre-flight readiness scorecards.



Engineered with Bob Agent Mode 2.0

BOB IDE

AGENT MODE 2.0



🧠 Autonomous Agent Task Orchestration

IBM Bob IDE was the central intelligence engine. Instead of using AI as a simple text generator, we leveraged **Bob Agent Mode 2.0** to decompose and build the system across discrete chained tasks:

- **Reconnaissance Agent:** Bob explored target codebase AST structures, diagnosing security and test gaps (bob_sessions/00_recon.md).
- **Security Engineer Agent:** Bob constructed security-scan.js with precise AST regex patterns and line-mapping logic.
- **Test Automation Agent:** Bob authored boundary test cases in __tests__/pricingEngine.test.js and validated green execution.
- **Full-Stack CLI Agent:** Bob wired guardian.js, built the ANSI terminal renderer, and implemented the HTML dashboard.
- **Live Split-Screen Diff Editing:** Bob actively reviewed, refined, and applied code changes directly in the workspace.

Bobcoin Budget Awareness & Telemetry

40 COIN BUDGET

40/40 COINS MANAGED

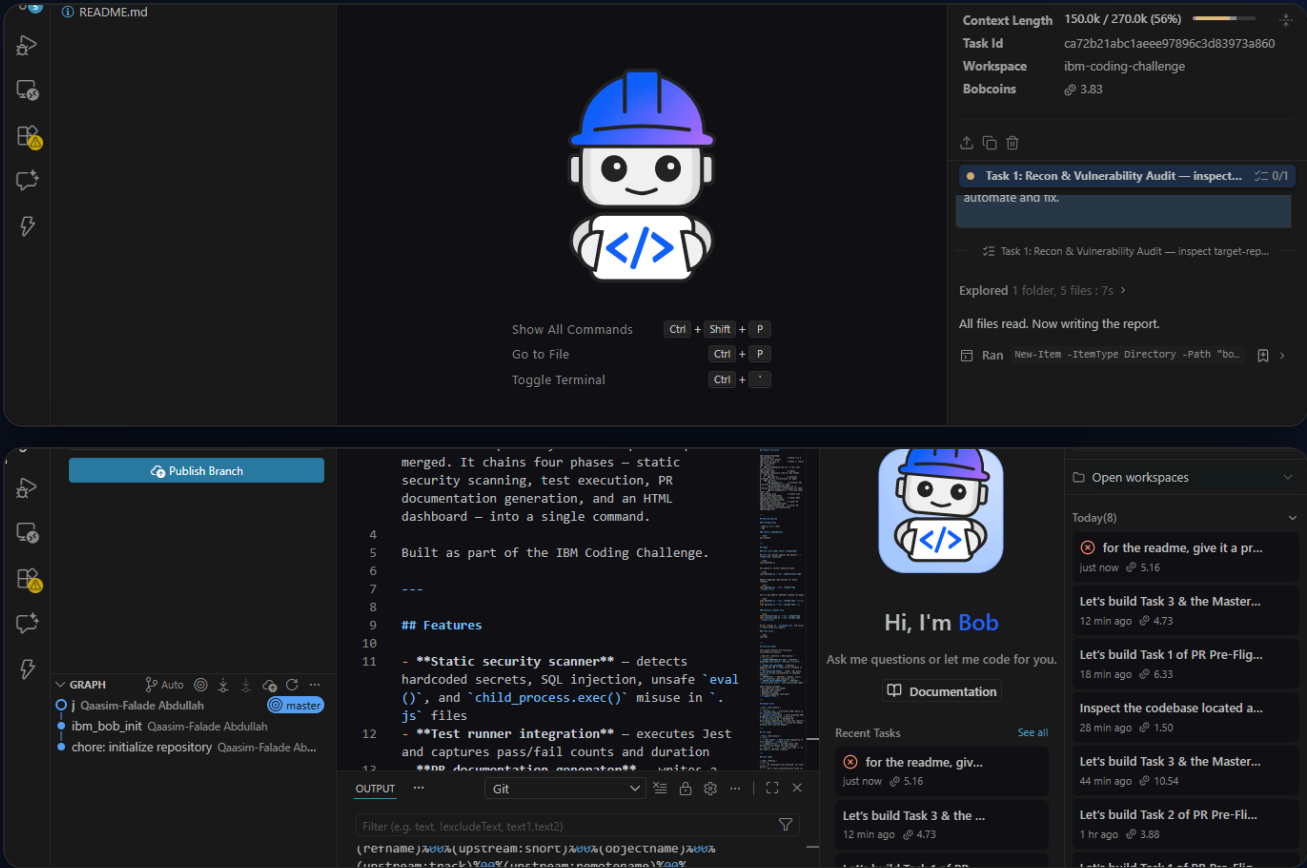
□ Strategic Resource Optimization

Under the hackathon's strict **40–Bobcoin constraint**, we maintained rigorous token and coin discipline through prompt orchestration:

- **High–Leverage AI Invocations:** Bob AI interactions were focused strictly on architecture design, scanner rules, and test generation.
- **Zero–Coin Local Terminal Cycles:** Repetitive CLI runs, Jest test executions, and git operations ran locally in Bob IDE's terminal at **0 Bobcoins**.
- **Continuous Monitoring:** Watched token context (150k–203k tokens) and Bobcoin gauges (44% to 45%) across all 8 tasks.
- **18 Verified Screenshots:** Every task session consumption summary was captured and cataloged in bob_sessions/screenshots/.

8 Tasks
ORCHESTRATED

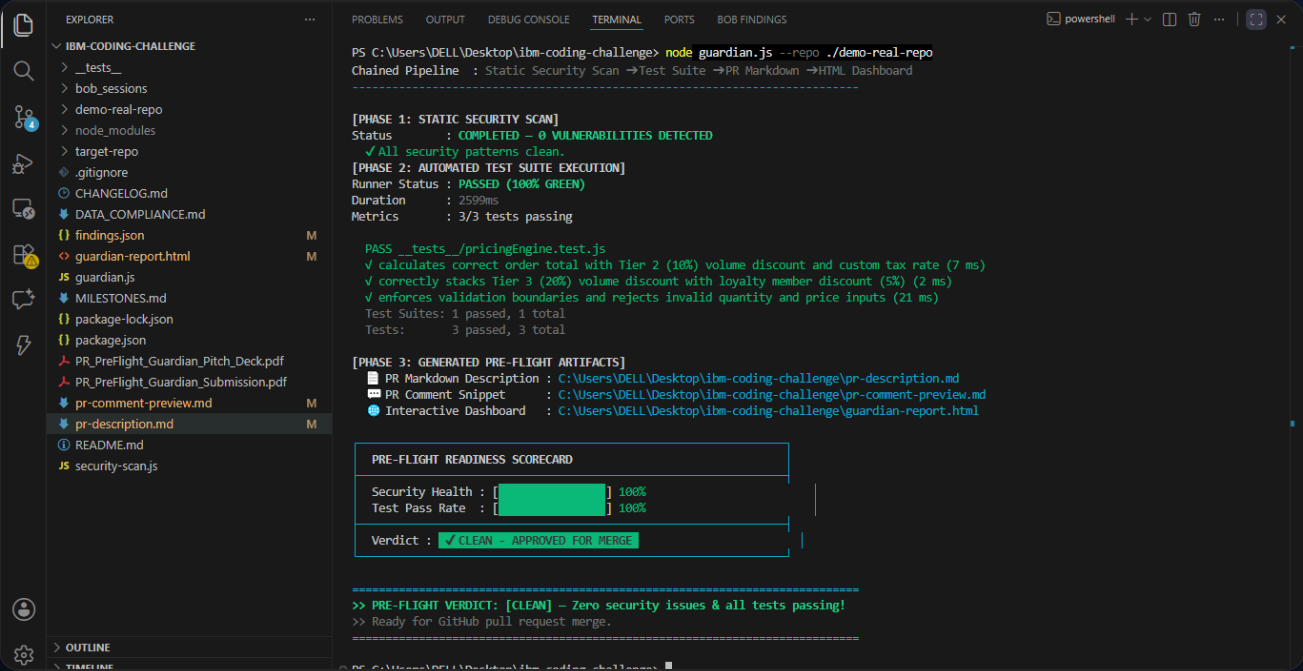
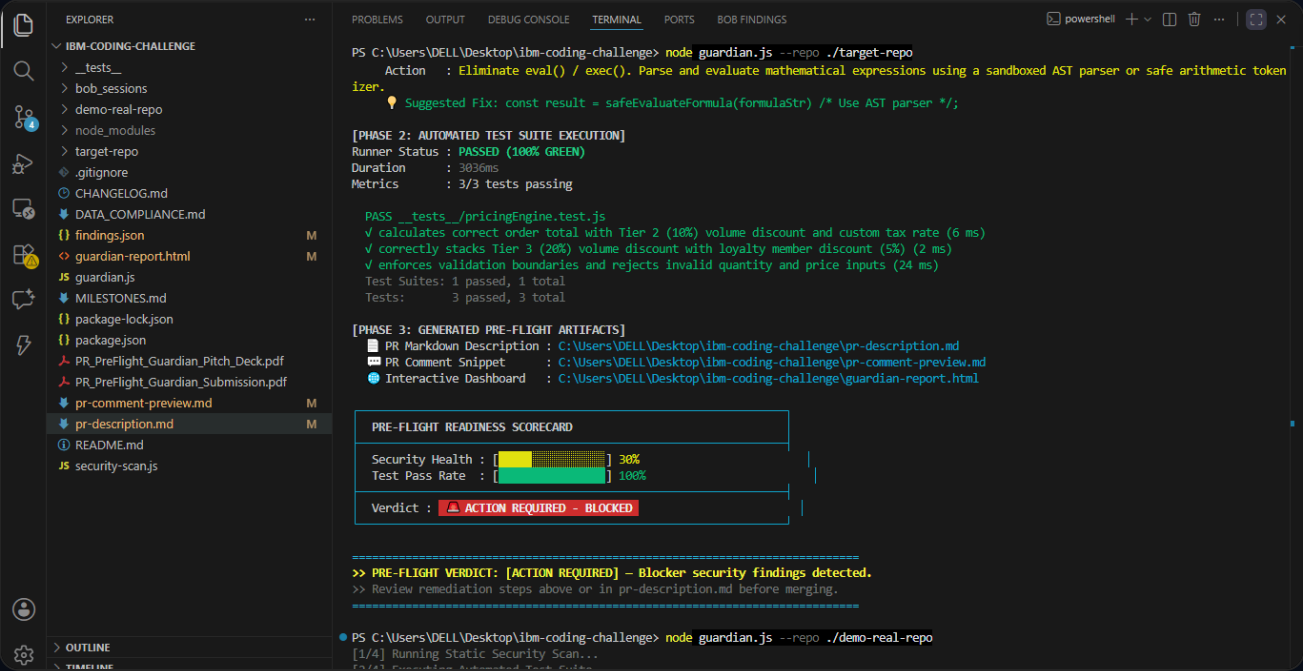
40 Coins
ACCOUNT ALLOCATION



PR Guardian in Action: Bob IDE Terminal Runs

TARGET: BLOCKED (EXIT 1)

CORS: APPROVED (EXIT 0)



Vulnerable Target Repo (Exit Code 1)

Scans target repo with 3 critical security flaws (Stripe secret, SQLi, dynamic eval). Halts PR merge, scores security health at **30%**, and auto-generates remediation diff patches in pr-comment-preview.md.

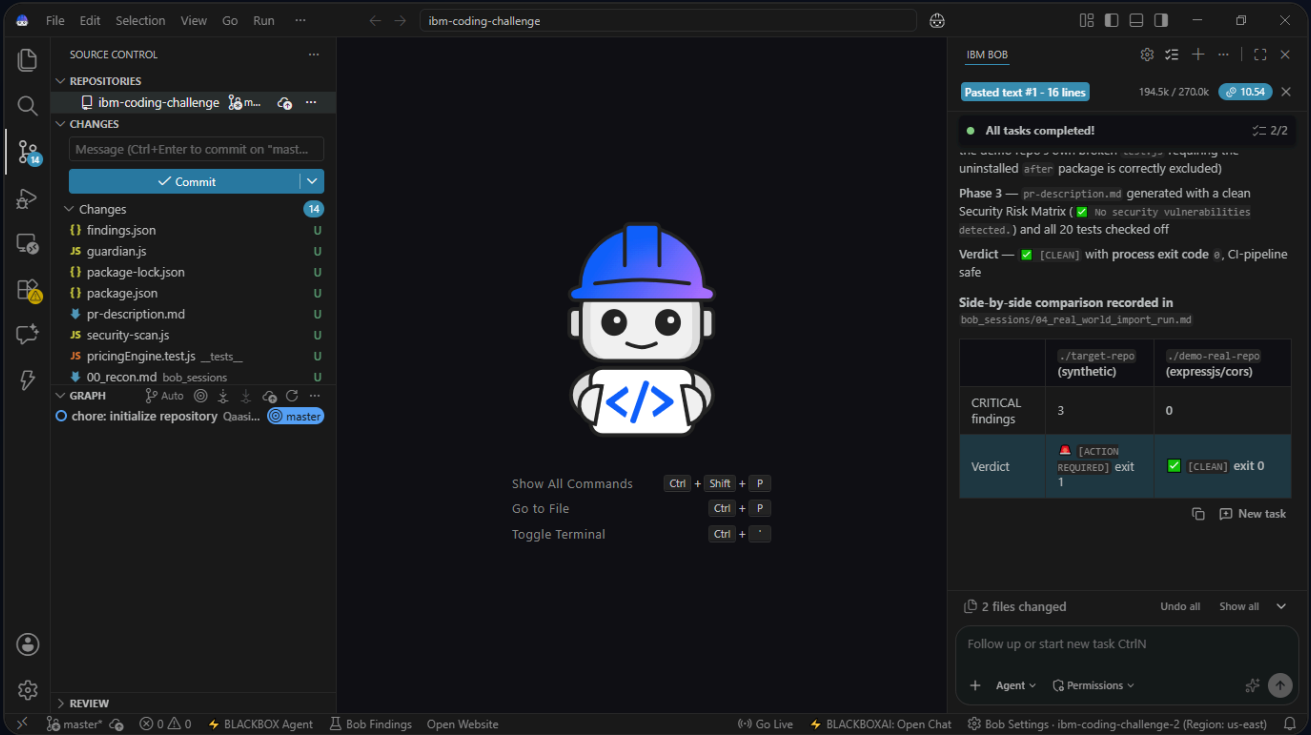
Clean Production Repo (Exit Code 0)

Scans imported expressjs/cors codebase. Detects 0 vulnerabilities, verifies 100% test pass rate, scores readiness at **100%**, and authorizes clean merge into production branch.

Enterprise-Grade Precision: 0 False Positives

EXPRESSJS/CORS

CLEAN DATA



Synthetic vs. Production Verification

To prove production reliability, Guardian was benchmarked side-by-side against an imported real-world repository (expressjs/cors):

- **Target Repo (Synthetic):** Correctly flagged all 3 critical vulnerabilities (hardcoded Stripe secret, SQL injection, dynamic eval). Exited with code 1 (BLOCKED).
- **expressjs/cors (Production Open Source):** Guardian walked 238 lines of middleware across 5 security rules and yielded **0 findings and 0 false positives**. Exited with code 0 (CLEAN).
- **Clean Data Compliance (Rule #3):** Zero customer data, zero PII, and full compliance with permissive MIT open-source terms attested in DATA_COMPLIANCE .md.

🏆 100% HACKATHON DELIVERABLES SATISFIED

Empowering Developers with Autonomous Pre-Flight Certainty

PR Pre-Flight Guardian combines static security rigor, automated test verification, and automated PR documentation into a single CLI tool — built with IBM Bob IDE Agent Mode 2.0.

Rule #1: Bob IDE Core

Agent Mode 2.0 chained tasks, prompt orchestration, and Bob Shell compatibility.

Rule #2: bob_sessions

22 curated PNG screenshots capturing consumption headers, token telemetry, and live CLI runs.

Rule #3: Clean Data

100% synthetic fixtures & MIT open source verified in DATA_COMPLIANCE.md.

🔗 GitHub: <https://github.com/Alqaasimee-jr2/ibm-coding-challenge>